

Autonomous QA Session — Design Specification

Date: 2026-03-19 **Status:** Approved **Author:** Claude Code + Milos Vasic

1. Overview

Extend HelixQA with a new test type called **Autonomous QA Session** that uses LLMs and computer vision to autonomously navigate applications, verify documented features, discover bugs, and generate comprehensive QA reports with video evidence.

Scope

- 3 new standalone Go modules (Git submodules): LLMOrchestrator, VisionEngine, DocProcessor
- LLMsVerifier Strategy pattern extension (existing repo)
- 4 new HelixQA packages + 6 enhanced existing packages
- ~1,100 new tests (unit, integration, E2E, security, stress, automation)
- 50 documentation files, 28 video course scripts, 25 Mermaid diagrams
- GitHub/GitLab project management with task tracking

Module Dependency Order

```
LLMsVerifier (no project deps)
  → DocProcessor (no project deps)
  → LLMOrchestrator (no project deps; accepts ModelInfo structs passed by caller)
  → VisionEngine (no project deps; VisionProvider interface calls LLM APIs directly)
  → HelixQA (depends on all above + Challenges + Containers)
```

Important: No Go module-level imports exist between LLMsVerifier, LLMOrchestrator, VisionEngine, and DocProcessor. Each defines its own interfaces. HelixQA is the integration point that wires them together via adapter implementations.

Repository Layout

Module	GitHub	GitLab	Go Module Path
LLMsVerifier	vasic-digital + HelixDevelopment	vasic-digital + HelixDevelopment	digital.vasic.llmsverifier
LLMOrchestrator	vasic-digital + HelixDevelopment	vasic-digital + HelixDevelopment	digital.vasic.llmorchestrator
VisionEngine	vasic-digital + HelixDevelopment	vasic-digital + HelixDevelopment	digital.vasic.visionengine

Module	GitHub	GitLab	Go Module Path
DocProcessor	vasic-digital + HelixDevelopment	vasic-digital + HelixDevelopment	<code>digital.vasic.docprocessor</code>
HelixQA	vasic-digital + HelixDevelopment	vasic-digital + HelixDevelopment	<code>digital.vasic.helixqa</code>

Go module paths follow the `digital.vasic.*` convention used by existing modules (Challenges, Containers, HelixQA). GitHub/GitLab remotes are wired via `replace` directives in `go.mod` during development.

All repos mirrored to 4 remotes (vasic-digital GitHub/GitLab + HelixDevelopment GitHub/GitLab).

2. Architecture

Overall Pattern: Coordinator + Worker Agents

A `SessionCoordinator` manages 4 sequential phases, delegating platform testing to parallel `PlatformWorker` instances. Each worker gets its own LLM agent, vision analyzer, navigation graph, and crash detector.

```
SessionCoordinator
├─ Phase 1: Setup (sequential)
│   ├── LLMsVerifier → select LLMs with QAStrategy
│   ├── DocProcessor → build feature map from project docs
│   └── LLMOrchestrator → spawn CLI agents
├─ Phase 2: Doc-Driven Verification (parallel per platform)
│   ├── Worker[Android] → verify documented features
│   ├── Worker[Desktop] → verify documented features
│   └── Worker[Web] → verify documented features
├─ Phase 3: Curiosity-Driven Exploration (parallel per platform)
│   ├── Worker[Android] → free exploration + edge cases
│   ├── Worker[Desktop] → free exploration + edge cases
│   └── Worker[Web] → free exploration + edge cases
└─ Phase 4: Report & Cleanup (sequential)
    └─ Aggregate tickets + evidence + coverage → QA report
```

Communication Model: Hybrid Pipe + File

- **Stdin/stdout pipes** for real-time interaction (navigation commands, screen analysis, quick decisions) using JSON-lines protocol
- **File-based exchange** for larger artifacts (screenshots, analysis reports, ticket drafts) using inbox/outbox/shared directories

3. LLMsVerifier Strategy Extension

New Interfaces

```
// ScoringStrategy – pluggable scoring mechanism
type ScoringStrategy interface {
    Name() string
    Description() string
    WeightConfig() WeightConfig
    CustomTests() []VerificationTest
    ScoreModel(ctx context.Context, model ModelInfo, results []TestResult)
    (StrategyScore, error)
    FilterModels(models []ModelInfo) []ModelInfo
    MinimumThresholds() Thresholds
}

// VerificationTest – individual test that a strategy can inject
type VerificationTest interface {
    ID() string
    Name() string
    Category() TestCategory
    Run(ctx context.Context, client *LLMClient) (TestResult, error) //
    *LLMClient is existing struct
    Weight() float64
    Required() bool // fail-fast if required test fails
}

// RecipeStep – composable building block for StrategyBuilder
type RecipeStep interface {
    Apply(builder *StrategyBuilder) *StrategyBuilder
}
```

New Types (Supporting the Strategy Interfaces)

```
// WeightConfig – extends existing ScoreWeights with new dimensions.
// Maps to the existing PerformanceScore fields for DefaultStrategy,
// adds VisionCapability and InstructionFollowing for custom strategies.
type WeightConfig struct {
    Responsiveness      float64 // maps to existing ScoreWeights.Responsiveness
    CodeCapability      float64 // maps to existing ScoreWeights.CodeCapability
    FeatureRichness     float64 // maps to existing
    ScoreWeights.FeatureRichness
    Reliability         float64 // maps to existing ScoreWeights.Reliability
    VisionCapability    float64 // NEW: weight for vision test results
    InstructionFollowing float64 // NEW: weight for instruction-following
    results
}

// Validate ensures weights sum to 1.0 (within float tolerance of 0.001)
func (wc WeightConfig) Validate() error

// TestCategory classifies verification tests
type TestCategory string
```

```

const (
    TestCategoryExistence      TestCategory = "existence"
    TestCategoryResponsiveness TestCategory = "responsiveness"
    TestCategoryLatency        TestCategory = "latency"
    TestCategoryFeature        TestCategory = "feature"
    TestCategoryCode           TestCategory = "code"
    TestCategoryVision         TestCategory = "vision"           // NEW
    TestCategoryInstruction     TestCategory = "instruction"     // NEW
    TestCategoryStability      TestCategory = "stability"       // NEW
)

// TestResult – outcome of a single VerificationTest run
type TestResult struct {
    TestID      string
    Category    TestCategory
    Passed      bool
    Score       float64          // 0.0 - 1.0
    Latency     time.Duration
    Error       error
    Details     map[string]any
    Timestamp   time.Time
}

// StrategyScore – returned by ScoringStrategy.ScoreModel().
// Contains the strategy-specific overall score plus a breakdown
// that can be converted to the existing PerformanceScore for backward compat.
type StrategyScore struct {
    Overall      float64          // 0.0 - 1.0
    Breakdown    map[string]float64 // dimension name → score
    Passed       bool              // meets minimum thresholds
    Details      map[string]any     // strategy-specific details
}

// ToPerformanceScore converts StrategyScore to the existing PerformanceScore
type
// for backward compatibility with existing report generation and API
responses.
func (ss StrategyScore) ToPerformanceScore() PerformanceScore

// Thresholds – minimum requirements for a model to be considered viable
type Thresholds struct {
    MinOverallScore float64
    MaxLatency      time.Duration
    MinContextWindow int
    RequiredCapabilities []string
}

```

StrategyBuilder (Fluent API)

```

strategy := NewStrategyBuilder("helix-qa").
    WithDescription("Optimized for autonomous QA sessions").

```

```
WithWeights(WaitConfig{
  Responsiveness:      0.15,
  CodeCapability:      0.10,
  FeatureRichness:     0.15,
  Reliability:         0.20,
  VisionCapability:     0.25, // NEW
  InstructionFollowing: 0.15, // NEW
}).
WithRecipe(recipes.VisionTest()).
WithRecipe(recipes.InstructionFollowingTest()).
WithRecipe(recipes.ContextWindowTest(minTokens: 100000)).
WithRecipe(recipes.StreamingReliabilityTest()).
WithRecipe(recipes.LongSessionStabilityTest(duration: 30*time.Minute)).
WithTest(&NavigationCommandTest{}).
RequireCapability("vision").
RequireCapability("streaming").
MinContextWindow(50000).
MinScore(0.6).
MaxLatency(10 * time.Second).
Build()
```

Built-in Recipes

Recipe	Purpose
VisionTest()	Screenshot analysis capability
InstructionFollowingTest()	Action precision measurement
ContextWindowTest(minTokens)	Context window validation
StreamingReliabilityTest()	Streaming consistency
LongSessionStabilityTest(duration)	Multi-hour session stability

Backward Compatibility

- **Verify() unchanged** — continues to use hardcoded weights and existing CalculateScores() logic. No signature or behavior change.
- **CalculateScores() unchanged** — retains its existing signature (VerificationResult (PerformanceScore, ScoreDetails)). No delegation.
- **New VerifyWithStrategy(ctx, strategy ScoringStrategy)** — parallel entry point that runs the strategy's custom tests and uses strategy.ScoreModel() instead of the hardcoded scoring. Returns (VerificationResult, []StrategyScore, error).
- **New CalculateScoresWithStrategy(result, strategy)** — strategy-aware scoring that returns []StrategyScore. Callers can convert via StrategyScore.ToPerformanceScore() for compatibility.
- **DefaultStrategy** — wraps current behavior exactly: same weights (0.30/0.25/0.25/0.20), same tests, same thresholds. DefaultStrategy.ScoreModel() produces identical results to CalculateScores().
- All current tests pass unchanged — no existing function signatures modified.

File Structure

```

llm-verifier/llmverifier/
├─ strategy.go           // ScoringStrategy interface
├─ strategy_default.go   // DefaultStrategy (wraps current behavior)
├─ strategy_builder.go   // StrategyBuilder fluent API
├─ strategy_test.go
├─ strategy_builder_test.go
├─ strategy_integration_test.go
├─ recipes/
│   ├── recipe.go        // RecipeStep interface + base recipes
│   ├── vision.go        // VisionTest recipe
│   ├── instruction.go    // InstructionFollowingTest
│   ├── context_window.go // ContextWindowTest
│   ├── streaming.go      // StreamingReliabilityTest
│   ├── long_session.go   // LongSessionStabilityTest
│   ├── recipes_test.go
│   └─ recipes_stress_test.go
├─ verifier.go           // MODIFIED: accepts ScoringStrategy
├─ models.go             // MODIFIED: WeightConfig extended
└─ scoring.go            // MODIFIED: delegates to strategy

```

4. LLMOrchestrator Module

Standalone Go module for managing headless CLI agents with hybrid pipe+file communication.

Core Interfaces

```

// Agent – headless CLI agent abstraction
type Agent interface {
    ID() string
    Name() string // "opencode", "claude-code", "gemini", "junie", "qwen-code"
    Start(ctx context.Context) error
    Stop(ctx context.Context) error
    IsRunning() bool
    Health(ctx context.Context) HealthStatus
    // Real-time interaction (stdin/stdout pipe)
    Send(ctx context.Context, prompt string) (Response, error)
    SendStream(ctx context.Context, prompt string) (<-chan StreamChunk, error)
    // File-based exchange (artifacts)
    SendWithAttachments(ctx context.Context, prompt string, attachments
[]Attachment) (Response, error)
    OutputDir() string
    // Capabilities
    Capabilities() AgentCapabilities
    SupportsVision() bool
    ModelInfo() ModelInfo
}

```

```
// AgentPool – thread-safe agent pool with capability matching
type AgentPool interface {
    Register(agent Agent) error
    Acquire(ctx context.Context, requirements AgentRequirements) (Agent, error)
    Release(agent Agent)
    Available() []Agent
    HealthCheck(ctx context.Context) []HealthStatus
    Shutdown(ctx context.Context) error
}

// ResponseParser – structured extraction from LLM output
type ResponseParser interface {
    Parse(raw string) (ParsedResponse, error)
    ExtractJSON(raw string) (map[string]any, error)
    ExtractActions(raw string) ([]Action, error)
    ExtractIssues(raw string) ([]Issue, error)
}
```

Supporting Types

```
// Response – parsed result from an Agent.Send() call
type Response struct {
    Content      string           // raw text response
    Actions      []Action        // extracted structured actions
    Metadata     map[string]string // provider-specific metadata
    TokensUsed   int             // total tokens consumed
    Latency      time.Duration   // round-trip time
    Error        error           // nil if successful
}

// StreamChunk – individual chunk from Agent.SendStream()
type StreamChunk struct {
    Content string
    Done    bool
    Error   error
}

// Attachment – file sent with SendWithAttachments
type Attachment struct {
    Path      string // file path
    MimeType  string // e.g., "image/png"
    Name      string // display name
}

// AgentCapabilities – what an agent can do
type AgentCapabilities struct {
    Vision      bool
    Streaming   bool
    ToolUse     bool
    MaxTokens   int
    MaxImages   int
}
```

```

    Providers []string // which LLM providers this agent supports
}

// AgentRequirements – what a caller needs from an agent
type AgentRequirements struct {
    NeedsVision    bool
    NeedsStreaming bool
    MinTokens      int
    PreferredAgent string // optional: prefer specific agent name
}

// HealthStatus – agent health check result
type HealthStatus struct {
    AgentID      string
    AgentName    string
    Healthy      bool
    Latency      time.Duration
    Error        error
    CheckedAt    time.Time
}

// Action – structured action extracted from LLM response
type Action struct {
    Type        string // "click", "type", "scroll", "navigate", "back", "home"
    Target      string // element label or coordinates
    Value       string // text to type, scroll amount, etc.
    Confidence  float64
}

// ParsedResponse – fully parsed agent response
type ParsedResponse struct {
    Raw        string
    Content    string
    Actions    []Action
    Issues     []Issue
    JSON       map[string]any // if response contained JSON
}

// Issue – problem detected by an agent
type Issue struct {
    Type        string // "visual", "ux", "accessibility", "functional",
    "performance", "crash"
    Severity    string // "critical", "high", "medium", "low"
    Title       string
    Description  string
    ScreenID    string
    Evidence    []string // screenshot paths
}

// ModelInfo – LLM model information (passed from LLMsVerifier via HelixQA)
type ModelInfo struct {
    ID          string
    Provider    string
    Name        string

```

```
Capabilities AgentCapabilities
Score         float64 // from LLMsVerifier
}
```

5 CLI Adapters

Adapter	CLI	Headless Mode	Vision
OpenCodeAgent	opencode	--headless --non-interactive	via configured LLM
ClaudeCodeAgent	claude	--print --output-format json	Claude vision (native)
GeminiAgent	gemini	--non-interactive	Gemini vision (native)
JunieAgent	junie	--headless	via configured LLM
QwenCodeAgent	qwen-code	--headless --non-interactive	Qwen-VL (native)

All adapters extend `BaseAdapter` which handles shared process management (spawn, pipe setup, graceful shutdown, timeout). Each adapter only implements protocol-specific parsing.

Communication Protocol

Pipe (real-time):

```
→ {"type": "prompt", "content": "What do you see on this
screen?", "image_path": "/tmp/screenshot.png"}
← {"type": "response", "content": "I see a login form...", "actions":
[{"type": "click", "target": "username_field"}]}
```

File (artifacts):

```
/tmp/helix-session-{id}/
├─ inbox/      ← HelixQA writes instructions
├─ outbox/     ← Agent writes results
└─ shared/    ← Shared artifacts (docs, feature map)
```

Key Design Decisions

- **BaseAdapter** — shared process management; each CLI adapter only implements protocol-specific parsing
- **AgentPool** — mutex-safe, capability-based matching via `Acquire(ctx, requirements)`
- **Circuit breaker per agent** — 3 consecutive crashes marks agent unhealthy
- **No hard dependency on LLMsVerifier** — receives `ModelInfo` from HelixQA, keeping modules decoupled

File Structure

```

LLMOrchestrator/
├─ cmd/orchestrator/main.go
├─ pkg/
│   ├─ agent/      (agent.go, pool.go, health.go + tests)
│   ├─ adapter/    (base.go, opencode.go, claudecode.go, gemini.go, junie.go,
qwencode.go + tests)
│   ├─ protocol/   (pipe.go, file.go, message.go + tests)
│   ├─ parser/     (parser.go, action.go + tests)
│   └─ config/     (config.go + tests)
├─ go.mod, Makefile, .env.example
├─ README.md, ARCHITECTURE.md, API_REFERENCE.md
├─ USER_GUIDE.md, CONTRIBUTING.md, CHANGELOG.md, LICENSE
└─ Upstreams/      (4 remote sync scripts)

```

5. VisionEngine Module

GoCV for mechanical vision + LLM Vision APIs for intelligent UI understanding.

No Go module dependency on LLMOrchestrator. VisionEngine defines its own `VisionProvider` interface. The `llmvision/` adapters (`openai.go`, `anthropic.go`, `gemini.go`, `qwen.go`) call LLM APIs directly via HTTP — they do NOT go through LLMOrchestrator's Agent pipe protocol. HelixQA passes API keys and provider configuration when constructing VisionEngine.

NavigationGraph is defined here and imported by HelixQA. HelixQA's `pkg/navigator/NavigationEngine` holds a `graph.NavigationGraph` from this module.

Core Interfaces

```

// Analyzer – primary vision analysis interface
type Analyzer interface {
    AnalyzeScreen(ctx context.Context, screenshot []byte) (ScreenAnalysis,
error)
    CompareScreens(ctx context.Context, before, after []byte) (ScreenDiff,
error)
    DetectElements(screenshot []byte) ([]UIElement, error)
    DetectText(screenshot []byte) ([]TextRegion, error)
    IdentifyScreen(ctx context.Context, screenshot []byte) (ScreenIdentity,
error)
    DetectIssues(ctx context.Context, screenshot []byte) ([]VisualIssue, error)
}

// NavigationGraph – directed graph of screens and transitions
type NavigationGraph interface {
    AddScreen(screen ScreenIdentity) string
    AddTransition(from, to string, action Action)
    CurrentScreen() string
    SetCurrent(screenID string)
    PathTo(targetID string) ([]Transition, error)
    UnvisitedScreens() []string
}

```

```

    Coverage() float64
    Export() GraphSnapshot
    Screens() []ScreenNode
    Transitions() []Transition
}

// VideoProcessor – GoCV-based video analysis
type VideoProcessor interface {
    ExtractFrame(videoPath string, timestamp time.Duration) ([]byte, error)
    ExtractKeyFrames(videoPath string) ([]KeyFrame, error)
    DetectSceneChanges(videoPath string) ([]time.Duration, error)
    GenerateThumbnail(videoPath string, ts time.Duration, size Size) ([]byte,
error)
}

// VisionProvider – LLM vision API adapter
type VisionProvider interface {
    AnalyzeImage(ctx context.Context, image []byte, prompt string) (string,
error)
    CompareImages(ctx context.Context, img1, img2 []byte, prompt string)
(string, error)
    SupportsVision() bool
    MaxImageSize() int
}

```

Two-Layer Analysis Pipeline

1. Layer 1: GoCV (mechanical) — fast, deterministic, no API cost

- Screenshot diffing (SSIM + pixel diff → change mask)
- Edge detection (Canny edges → UI element bounding boxes)
- Color analysis (dominant colors, contrast ratios)
- Template matching (find known icons/buttons)
- Contour detection (bounding boxes for all UI elements)
- Video frame extraction at scene changes

2. Layer 2: LLM Vision (intelligent) — contextual, semantic

- Screen identification ("This is the Settings page")
- UI comprehension ("Login form with email + password")
- UX issue detection ("Button text is truncated")
- Accessibility analysis ("Low contrast on header text")
- Navigation suggestions ("Click hamburger menu → Settings")
- Feature verification ("Markdown preview matches docs")

GoCV runs first → structured context fed to LLM Vision (e.g., "14 detected elements at these coordinates. Which is the Settings button?")

Key Data Types

```

type ScreenAnalysis struct {
    ScreenID    string
    Title       string          // LLM-identified screen name
    Description string
    Elements    []UIElement
    TextRegions []TextRegion
    Issues      []VisualIssue
    Navigable   []Action          // possible navigation actions
    Timestamp   time.Time
}

type UIElement struct {
    Type        string // button, input, link, menu, tab, etc.
    Label       string
    BoundingBox Rect
    Clickable   bool
    Confidence   float64 // 0-1
}

type ScreenDiff struct {
    Similarity      float64 // SSIM score 0-1
    ChangedRegions []Rect
    NewElements     []UIElement
    GoneElements    []UIElement
    DiffImage       []byte
    IsNewScreen     bool
}

```

LLM Vision Providers

4 adapters: GPT-4o (OpenAI), Claude (Anthropic), Gemini (Google), Qwen-VL. Provider selected based on LLMsVerifier scores with fallback chain.

NavigationGraph

Directed graph where nodes are unique screens (hashed by visual similarity) and edges are actions. Supports BFS pathfinding. Exports to DOT (Graphviz), JSON, Mermaid.

File Structure

```

VisionEngine/
├─ pkg/
│  ├─ analyzer/   (analyzer.go, screen.go, element.go + tests)
│  ├─ opencv/     (differ.go, detector.go, color.go, video.go + tests)
│  └─ llmvision/  (provider.go, openai.go, anthropic.go, gemini.go, qwen.go +
tests)
│  ├─ graph/      (graph.go, pathfinder.go, export.go + tests)
│  └─ config/     (config.go + tests)
├─ go.mod (depends on gocv.io/x/gocv)
├─ Makefile, .env.example

```

```
├ README.md, ARCHITECTURE.md, API_REFERENCE.md
├ USER_GUIDE.md, CONTRIBUTING.md, CHANGELOG.md, LICENSE
└ Upstreams/
```

6. DocProcessor Module

Loads project documentation, builds structured feature maps, tracks verification coverage.

Core Interfaces

```
// Loader – document loading and parsing
type Loader interface {
    LoadDir(ctx context.Context, path string) ([]Document, error)
    LoadFile(ctx context.Context, path string) (Document, error)
    SupportedFormats() []string
}

// FeatureMapBuilder – builds feature maps from documents using LLM
type FeatureMapBuilder interface {
    BuildFromDocs(ctx context.Context, docs []Document) (*FeatureMap, error)
    Enrich(ctx context.Context, fm *FeatureMap, agent LLMAgent) error
    Merge(maps ...*FeatureMap) *FeatureMap
}

// CoverageTracker – thread-safe feature verification tracking
type CoverageTracker interface {
    MarkVerified(featureID string, platform string, evidence Evidence)
    MarkFailed(featureID string, platform string, issue Issue)
    MarkSkipped(featureID string, platform string, reason string)
    Coverage() CoverageReport
    CoverageByPlatform(platform string) float64
    CoverageByCategory(category string) float64
    Unverified() []Feature
    Export() CoverageSnapshot
}

// LLMAgent – injected LLM interface (no dependency on LLMOrchestrator)
type LLMAgent interface {
    Summarize(ctx context.Context, text string) (string, error)
    ExtractFeatures(ctx context.Context, text string) ([]RawFeature, error)
    ClassifyFeature(ctx context.Context, feature RawFeature) (FeatureCategory, error)
    InferScreens(ctx context.Context, features []Feature) ([]ExpectedScreen, error)
    GenerateTestSteps(ctx context.Context, feature Feature) ([]TestStep, error)
}
```

Processing Pipeline

1. **Load & Parse** — scan project tree for docs (*.md, *.yaml, *.html, *.adoc, *.rst). Auto-discover by well-known patterns (docs/, README.md, *_GUIDE.md, ARCHITECTURE.md).
2. **Extract Features (LLM)** — LLM reads each document, extracts features, screens, workflows with structured JSON output.
3. **Build Feature Map** — structured, queryable map: Features[], Screens[], Workflows[], Categories{}, PlatformMatrix{}, DocGraph.
4. **Generate Verification Plan** — LLM generates test steps for each feature, prioritized by risk.

Key Data Types

```

type Feature struct {
    ID            string          // deterministic: "feat-markdown-editing"
    Name          string
    Description   string
    Category      FeatureCategory // format, ui, network, settings, etc.
    Platforms     []string
    Priority       string          // critical, high, medium, low
    Screens       []string        // expected screen IDs
    TestSteps     []TestStep
    SourceDoc     string
    SourceSection string
}

type FeatureMap struct {
    Features      []Feature
    Screens       []ExpectedScreen
    Workflows     []Workflow
    Categories     map[FeatureCategory][]Feature
    PlatformMatrix map[string][]Feature
    DocGraph      *DocGraph
    GeneratedAt   time.Time
    ProjectRoot   string
}

type CoverageReport struct {
    Total      int
    Verified   int
    Failed     int
    Skipped    int
    Unverified int
    OverallPct float64
    ByPlatform map[string]float64
    ByCategory map[string]float64
    Issues     []Issue
}

```

CoverageTracker Implementation

```
// coverageTracker — thread-safe implementation using sync.RWMutex
type coverageTracker struct {
    features map[string]*featureStatus // featureID → status per platform
    mu       sync.RWMutex
}

type featureStatus struct {
    FeatureID string
    Platforms map[string]VerificationStatus // platform → status
}

type VerificationStatus struct {
    State    string // "verified", "failed", "skipped", "unverified"
    Evidence Evidence // for verified
    Issue    Issue    // for failed
    Reason   string  // for skipped
}
```

Read operations (`Coverage()`, `CoverageByPlatform()`, `Unverified()`) use `mu.RLock()`. Write operations (`MarkVerified()`, `MarkFailed()`, `MarkSkipped()`) use `mu.Lock()`.

Key Design Decisions

- **LLMAgent is injected** — no module-level dependency on LLMOrchestrator. Fully reusable.
- **CoverageTracker is thread-safe** — `sync.RWMutex` protects all state; multiple platform workers mark features concurrently.
- **Feature IDs are deterministic** — same feature gets same ID across runs for progress comparison.
- **Prompt templates versioned** in `prompts.go` for trackable changes.

File Structure

```
DocProcessor/
├─ pkg/
│  ├─ loader/    (loader.go, markdown.go, yaml.go, scanner.go + tests)
│  ├─ feature/   (builder.go, feature.go, category.go, screen.go + tests)
│  ├─ coverage/  (tracker.go, report.go + tests)
│  ├─ docgraph/  (graph.go, export.go + tests)
│  ├─ llm/       (agent.go, prompts.go + tests)
│  └─ config/    (config.go + tests)
├─ go.mod (minimal deps: yaml.v3, testify)
├─ Makefile, .env.example
├─ README.md, ARCHITECTURE.md, API_REFERENCE.md
├─ USER_GUIDE.md, CONTRIBUTING.md, CHANGELOG.md, LICENSE
└─ Upstreams/
```

7. HelixQA — Autonomous QA Session

New Packages

pkg/autonomous — Session Coordinator

```

type SessionCoordinator struct {
    config      *SessionConfig
    verifier    LLMsVerifierClient
    docProcessor DocProcessorClient
    orchestrator AgentPool
    visionEngine Analyzer
    featureMap   *FeatureMap
    workers      map[string]*PlatformWorker
    phaseManager *PhaseManager
    session      *SessionRecorder
    mu           sync.Mutex
}

func (sc *SessionCoordinator) Run(ctx context.Context) (*SessionResult, error)
func (sc *SessionCoordinator) Pause(ctx context.Context) error
func (sc *SessionCoordinator) Resume(ctx context.Context) error
func (sc *SessionCoordinator) Cancel(ctx context.Context) error
func (sc *SessionCoordinator) Status() SessionStatus
func (sc *SessionCoordinator) Progress() ProgressReport

// PlatformWorker – executes both doc-driven and curiosity-driven phases for
one platform
type PlatformWorker struct {
    platform    string           // "android", "desktop", "web"
    agent       Agent           // acquired from AgentPool
    analyzer    Analyzer        // VisionEngine analyzer
    navigator   *NavigationEngine
    issueDetector *IssueDetector
    coverage    CoverageTracker // from DocProcessor
    navGraph    NavigationGraph // from VisionEngine/pkg/graph
    detector    CrashDetector   // existing HelixQA detector
    session     *SessionRecorder
    executor    ActionExecutor  // platform-specific (ADB/Playwright/X11)
    mu         sync.Mutex
}

func (pw *PlatformWorker) RunDocDriven(ctx context.Context, features []Feature)
([]StepResult, error)
func (pw *PlatformWorker) RunCuriosityDriven(ctx context.Context, timeout
time.Duration) ([]StepResult, error)

// PhaseManager – tracks phase transitions with listener notifications
type PhaseManager struct {
    phases    []Phase
    current   int
    listeners []PhaseListener
    mu       sync.Mutex
}

```

```

type Phase struct {
    Name      string      // "setup", "doc-driven", "curiosity", "report"
    Status    PhaseStatus // pending, running, completed, failed, skipped
    StartAt   time.Time
    EndAt     time.Time
    Progress  float64      // 0.0 - 1.0
    Error     error
}

type PhaseStatus string
const (
    PhasePending   PhaseStatus = "pending"
    PhaseRunning   PhaseStatus = "running"
    PhaseCompleted PhaseStatus = "completed"
    PhaseFailed    PhaseStatus = "failed"
    PhaseSkipped   PhaseStatus = "skipped"
)

// PhaseListener receives notifications on phase transitions
type PhaseListener interface {
    OnPhaseStart(phase Phase)
    OnPhaseComplete(phase Phase)
    OnPhaseError(phase Phase, err error)
}

func (pm *PhaseManager) Start(name string) error // pending → running
func (pm *PhaseManager) Complete(name string) error // running → completed
func (pm *PhaseManager) Fail(name string, err error) error
func (pm *PhaseManager) Skip(name string) error
func (pm *PhaseManager) Current() Phase
func (pm *PhaseManager) All() []Phase

```

pkg/navigator — Navigation Engine

```

type NavigationEngine struct {
    agent      Agent
    analyzer   Analyzer
    executor   ActionExecutor
    graph      NavigationGraph
    state      *StateTracker
}

func (ne *NavigationEngine) NavigateTo(ctx context.Context, target string)
error
func (ne *NavigationEngine) PerformAction(ctx context.Context, action Action)
(*ActionResult, error)
func (ne *NavigationEngine) ExploreUnknown(ctx context.Context)
(*ExploreResult, error)
func (ne *NavigationEngine) CurrentScreen(ctx context.Context)
(*ScreenAnalysis, error)

```

```

func (ne *NavigationEngine) GoBack(ctx context.Context) error
func (ne *NavigationEngine) GoHome(ctx context.Context) error

// ActionExecutor – platform-specific
type ActionExecutor interface {
    Click(ctx context.Context, x, y int) error
    Type(ctx context.Context, text string) error
    Scroll(ctx context.Context, direction string, amount int) error
    LongPress(ctx context.Context, x, y int) error
    Swipe(ctx context.Context, fromX, fromY, toX, toY int) error
    KeyPress(ctx context.Context, key string) error
    Back(ctx context.Context) error
    Home(ctx context.Context) error
    Screenshot(ctx context.Context) ([]byte, error)
}

// Implementations: ADBExecutor (Android), PlaywrightExecutor (Web),
X11Executor (Desktop)

```

pkg/issuedetector — LLM-Powered Bug Detection

```

type IssueDetector struct {
    agent      Agent
    analyzer   Analyzer
    ticketGen  *ticket.Generator
    session    *SessionRecorder
}

func (id *IssueDetector) AnalyzeAction(ctx, before, after ScreenAnalysis,
action Action) ([]Issue, error)
func (id *IssueDetector) AnalyzeUX(ctx, navGraph NavigationGraph) ([]Issue,
error)
func (id *IssueDetector) AnalyzeAccessibility(ctx, screen ScreenAnalysis)
([]Issue, error)
func (id *IssueDetector) CreateTicket(ctx, issue Issue) (*Ticket, error)

```

Issue categories: visual bugs, UX issues, accessibility, functional, performance, crashes.

pkg/session — Recording & Timeline

```

type SessionRecorder struct {
    sessionID      string
    outputDir      string
    videos         map[string]*VideoManager // platform → video
    timeline       *Timeline
    screenshotIdx  int
    mu             sync.Mutex
}

```

```

func (sr *SessionRecorder) StartRecording(ctx, platform string) error
func (sr *SessionRecorder) StopRecording(ctx, platform string) (string, error)
func (sr *SessionRecorder) CaptureScreenshot(ctx, platform, name string)
(Screenshot, error)
func (sr *SessionRecorder) RecordEvent(event TimelineEvent)
func (sr *SessionRecorder) VideoTimestamp(platform string) time.Duration
func (sr *SessionRecorder) ExportTimeline() []TimelineEvent

type TimelineEvent struct {
    ID          string
    Type        EventType // action, screenshot, issue, phase_change, crash,
navigation
    Platform    string
    Timestamp   time.Time
    VideoOffset time.Duration // offset into platform video
    ScreenID    string
    Description string
    ScreenshotPath string
    IssueID     string
    FeatureID   string
    Metadata    map[string]string
}

```

Video recording: ffmpeg (Desktop x11grab), adb screenrecord (Android), Playwright API (Web).

Bridge Adapters

HelixQA is the integration point that bridges the decoupled modules. Key adapter:

```

// agentLLMAdapter bridges LLMOrchestrator's Agent to DocProcessor's LLMAgent.
// Lives in HelixQA's pkg/autonomous/adapters.go
type agentLLMAdapter struct {
    agent agent.Agent // from LLMOrchestrator
    parser parser.ResponseParser
}

func (a *agentLLMAdapter) Summarize(ctx context.Context, text string) (string,
error) {
    resp, err := a.agent.Send(ctx, fmt.Sprintf("Summarize this
document:\n\n%s", text))
    if err != nil { return "", err }
    return resp.Content, nil
}

func (a *agentLLMAdapter) ExtractFeatures(ctx context.Context, text string)
([]RawFeature, error) {
    resp, err := a.agent.Send(ctx, featureExtractionPrompt(text))
    if err != nil { return nil, err }
    return a.parser.ExtractJSON(resp.Content) // parse structured JSON output
}

```

```
// ... similar for ClassifyFeature, InferScreens, GenerateTestSteps
```

Similarly, `VisionAgentAdapter` bridges LLMOrchestrator's Agent to VisionEngine's VisionProvider for cases where the same agent should be reused for both navigation and vision (optional — VisionEngine can also call APIs directly).

Agent-to-Platform Allocation

Each `PlatformWorker` acquires one dedicated `Agent` from the pool for the duration of its testing. The allocation strategy:

- If `HELIX_AGENT_POOL_SIZE >= len(platforms)`: one agent per platform, parallel execution
- If `HELIX_AGENT_POOL_SIZE < len(platforms)`: platforms queue for agents via `AgentPool.Acquire()` (blocks with context cancellation)
- Agent requirements: `NeedsVision: true` for all platform workers
- PreferredAgent: configurable per platform in `.env` (e.g., `HELIX_ANDROID_PREFERRED_AGENT=claude-code`)

Session Lifecycle

Phase 1: Setup (Sequential)

1. Load `.env` configuration
2. `LLMsVerifier.VerifyWithStrategy(ctx, QAStrategy)` → ranked models
3. `DocProcessor.LoadDir + BuildFromDocs + Enrich` → feature map
4. `LLMOrchestrator.SpawnAgents` → agent pool
5. `VisionEngine.Init` → analyzer
6. Start video recording per platform

Phase 2: Doc-Driven Verification (Parallel per Platform) For each feature (priority order):

- Agent receives feature + test steps
- Per step: capture pre-screenshot → agent analyzes → execute action → capture post-screenshot → diff → evaluate outcome → mark coverage → record timeline event
- Issue found → `IssueDetector` creates ticket
- `NavigationGraph` updated with each transition

Phase 3: Curiosity-Driven Exploration (Parallel per Platform)

- Agent receives `NavigationGraph` + "explore undiscovered areas"
- Loop: capture screen → agent decides action → execute → analyze result → update graph
- Edge case testing: empty inputs, rapid clicks, back button, rotation
- Undocumented features flagged for doc updates

Phase 4: Report & Cleanup (Sequential)

- Stop video recordings
- Aggregate coverage, tickets, navigation maps
- Link ticket screenshots to video timestamps

- Generate QA report (Markdown + HTML + JSON)
- Shutdown agent pool

Enhanced Ticket Format

```
# HQA-0042: Button text truncated on Android settings screen

**Severity:** Medium | **Platform:** Android | **Category:** Visual Bug

## Steps to Reproduce
1. Open app on Android device (Pixel 5)
2. Navigate: Home → Settings (hamburger menu)
3. Scroll down to "Data Management" section

## Evidence
- Screenshots: screenshots/android/042-settings-before.png, 042-annotated.png
- Video: videos/android-session.mp4 @ 14:32 (navigating), @ 14:47 (truncation visible)
- Logs: logs/android/042-logcat.txt

## LLM Analysis
Button uses fixed-width container (240dp). Recommended fix: wrap_content with minWidth.
```

Existing Package Enhancements

Package	Enhancement
pkg/detector	LLMCrashAnalyzer interface, root cause analysis
pkg/validator	LLMValidator for semantic pass/fail evaluation
pkg/evidence	LLM-annotated screenshots with bounding boxes
pkg/ticket	VideoReference struct, LLMSuggestedFix field
pkg/reporter	ExecutiveSummary, NavigationMapEmbed (Mermaid)
pkg/testbank	GenerateFromFeatureMap(), ExpandEdgeCases()

All enhancements are optional — without LLM agents, existing behavior preserved. Original `helixqa run` works identically. All 235 existing tests remain passing.

New HelixQA File Structure (additions)

```
HelixQA/pkg/
├─ autonomous/      (coordinator.go, worker.go, phase.go, result.go + 6 test files)
├─ navigator/        (engine.go, executor.go, adb.go, playwright.go, x11.go, state.go + 6 test files)
├─ issuedetector/    (detector.go, categories.go, prompts.go + 3 test files)
```

```
├ session/          (recorder.go, timeline.go, video.go + 4 test files)
├ config/           (MODIFIED)
├ evidence/         (MODIFIED)
├ ticket/           (MODIFIED)
├ reporter/         (MODIFIED)
├ orchestrator/     (MODIFIED)
└ detector/         (MODIFIED)
```

```
HelixQA/cmd/helixqa/
└ main.go           (MODIFIED – add "autonomous" subcommand)
```

CLI

```
helixqa autonomous --project /path/to/Yole \
  --platforms android,desktop,web \
  --env .env \
  --timeout 2h \
  --coverage-target 0.9 \
  --output qa-results/ \
  --report markdown,html,json
```

8. Configuration

Single `.env` file as source of truth. Autonomous session enabled by default, every aspect individually toggleable.

```
# Master Switch
HELIX_AUTONOMOUS_ENABLED=true
HELIX_AUTONOMOUS_PLATFORMS=android,desktop,web
HELIX_AUTONOMOUS_TIMEOUT=2h
HELIX_AUTONOMOUS_COVERAGE_TARGET=0.90
HELIX_AUTONOMOUS_CURIOSITY_ENABLED=true
HELIX_AUTONOMOUS_CURIOSITY_TIMEOUT=30m

# LLMsVerifier
LLMSVERIFIER_CONFIG=./llmsverifier.yaml
LLMSVERIFIER_STRATEGY=helix-qa
LLMSVERIFIER_MIN_SCORE=0.6
LLMSVERIFIER_MAX_MODELS=5
LLMSVERIFIER_CACHE_RESULTS=true
LLMSVERIFIER_CACHE_TTL=24h

# API Keys
OPENAI_API_KEY=sk-...
ANTHROPIC_API_KEY=sk-ant-...
GOOGLE_API_KEY=AI...
GROQ_API_KEY=gsk-...
MISTRAL_API_KEY=...
```

```
DEEPSEEK_API_KEY=...
XAI_API_KEY=...
TOGETHER_API_KEY=...
QWEN_API_KEY=...
JUNIE_API_KEY=...

# CLI Agents
HELIX_AGENTS_ENABLED=opencode, claude-code, gemini
HELIX_AGENT_OPENCODE_PATH=/usr/local/bin/opencode
HELIX_AGENT_CLAUDE_PATH=/usr/local/bin/claude
HELIX_AGENT_GEMINI_PATH=/usr/local/bin/gemini
HELIX_AGENT_JUNIE_PATH=/usr/local/bin/junie
HELIX_AGENT_QWEN_PATH=/usr/local/bin/qwen-code
HELIX_AGENT_TIMEOUT=60s
HELIX_AGENT_MAX_RETRIES=3
HELIX_AGENT_POOL_SIZE=3

# Vision Engine
HELIX_VISION_PROVIDER=auto
HELIX_VISION_OPENCV_ENABLED=true
HELIX_VISION_SSIM_THRESHOLD=0.95
HELIX_VISION_MAX_IMAGE_SIZE=4096

# Doc Processor
HELIX_DOCS_ROOT=./docs
HELIX_DOCS_AUTO_DISCOVER=true
HELIX_DOCS_FORMATS=md,yaml,html,adoc,rst

# Recording
HELIX_RECORDING_VIDEO=true
HELIX_RECORDING_SCREENSHOTS=true
HELIX_RECORDING_VIDEO_QUALITY=medium
HELIX_RECORDING_SCREENSHOT_FORMAT=png
HELIX_RECORDING_FFMPEG_PATH=/usr/bin/ffmpeg

# Platform-specific
HELIX_ANDROID_DEVICE=emulator-5554
HELIX_ANDROID_PACKAGE=digital.vasic.yole
HELIX_WEB_URL=http://localhost:8080
HELIX_WEB_BROWSER=chromium
HELIX_DESKTOP_PROCESS=yole-desktop
HELIX_DESKTOP_DISPLAY=:0

# Output
HELIX_OUTPUT_DIR=./qa-results
HELIX_REPORT_FORMATS=markdown,html,json
HELIX_TICKETS_ENABLED=true
HELIX_TICKETS_MIN_SEVERITY=low
```

9. Build Requirements & Platform Prerequisites

VisionEngine CGo Dependency

VisionEngine depends on GoCV ([gocv.io/x/gocv](https://github.com/gocv-io/x/gocv)) which requires OpenCV 4.x C libraries via CGo. To prevent this from contaminating HelixQA's core build:

- **Build tag `vision`** — all GoCV-dependent code is gated behind `//go:build vision`. Without this tag, VisionEngine compiles with stub implementations that return "OpenCV not available" errors. LLM Vision providers (which are pure Go HTTP) work without the tag.
- **HelixQA imports VisionEngine conditionally** — `pkg/autonomous/vision_opencv.go` (with `//go:build vision`) and `pkg/autonomous/vision_stub.go` (without). CI pipelines that don't have OpenCV installed can still build and test everything except OpenCV-specific features.
- **Docker image** — a `Dockerfile.vision` is provided with OpenCV 4.x pre-installed for full-featured builds.

Platform Prerequisites

Platform	Required Tools
Android	<code>adb</code> (Android SDK platform-tools), <code>scrcpy</code> (optional, for video)
Desktop	<code>ffmpeg</code> , <code>xdotool</code> or <code>xdo</code> (Linux), <code>import</code> from ImageMagick (screenshots)
Web	<code>playwright</code> (Node.js), <code>chromium</code> or <code>chrome</code> browser
All	<code>ffmpeg</code> (video recording/processing), Go 1.24+
Vision (optional)	OpenCV 4.x, pkg-config, GoCV build deps

Go Module Build

```
# Core build (no OpenCV needed)
go build ./...

# Full build with OpenCV vision
go build -tags vision ./...

# Tests (no OpenCV)
go test ./... -race -count=1

# Tests with vision features
go test -tags vision ./... -race -count=1
```

10. Resilience & Error Handling

LLM Failure Strategy

LLM calls are inherently unreliable. Every LLM interaction must handle:

- 1. **Retry policy:** Each LLM call type has a configured retry count (default: 3) with exponential backoff (1s, 2s, 4s). Configurable via `HELIX_AGENT_MAX_RETRIES`.
- 2. **Malformed JSON fallback:** When LLM returns invalid JSON (common for `ExtractFeatures`, `ExtractActions`):
 - Attempt regex extraction of JSON blocks from response
 - If still invalid, re-prompt with explicit JSON schema reminder
 - After 3 failed parse attempts, log warning and skip that item (don't crash the session)
- 3. **Vision provider fallback chain:** If primary `VisionProvider` fails, try next provider in score-ranked order. If all providers fail, fall back to GoCV-only analysis (mechanical vision without LLM understanding). Session continues with degraded but functional vision.
- 4. **Agent circuit breaker:** Per-agent circuit breaker (3 consecutive failures → open for 60s). Open circuit triggers:
 - `AgentPool` marks agent unhealthy
 - `PlatformWorker` acquires a replacement agent if available
 - If no agents available, session pauses and waits for recovery (up to `HELIX_AGENT_TIMEOUT`)
 - If timeout exceeded, phase fails gracefully with partial results
- 5. **Graceful degradation levels:**
 - **Full capability:** All LLM + Vision working → full autonomous session
 - **Degraded vision:** LLM Vision fails → GoCV-only (no semantic understanding, still navigates via element detection)
 - **Degraded navigation:** All agents fail → session collects whatever evidence it has and generates partial report
 - **Session abort:** Unrecoverable errors (no agents, no platforms) → clean shutdown with error report
- 6. **Prompt injection protection:** All LLM responses are sanitized before being used as file paths, shell commands, or ticket content. Path traversal patterns, shell metacharacters, and excessively long responses are rejected.

11. Testing Strategy

Coverage Requirements

~1,100 new tests across 6 mandatory test types per module:

Module	Unit	Integration	E2E	Security	Stress	Automation	Total
LLMsVerifier (Strategy)	✓	✓	✓	✓	✓	✓	~120
LLMOrchestrator	✓	✓	✓	✓	✓	✓	~200
VisionEngine	✓	✓	✓	✓	✓	✓	~180
DocProcessor	✓	✓	✓	✓	✓	✓	~150

Module	Unit	Integration	E2E	Security	Stress	Automation	Total
HelixQA (new)	✓	✓	✓	✓	✓	✓	~300
HelixQA (enhanced)	✓	✓	✓	✓	✓	✓	~150

Test Types

- **Unit (~400):** Every public function, table-driven edge cases, error paths, interface contracts
- **Integration (~250):** Cross-package interactions, real file I/O, pipeline flows
- **E2E (~150):** Full session lifecycle, CLI execution, mock LLM server
- **Security (~100):** API key masking, path traversal, command injection, prompt injection
- **Stress (~100):** Concurrent pool operations, large graphs, high-frequency capture, race detection
- **Automation (~100):** CI validation, clean build, cross-module deps, upstream sync

Test Infrastructure

- **Mock infrastructure:** MockAgent, MockVisionProvider, MockLLMServer, MockActionExecutor, TestFixtures
- **No real API keys or devices required** — all external dependencies mocked for CI
- **Naming convention** (applies to all new code; existing LLMsVerifier tests retain their current names):
*_test.go (unit), *_integration_test.go, *_e2e_test.go, *_security_test.go, *_stress_test.go, *_automation_test.go
- **All tests:** go test ./... -race -count=1 (add -tags vision for OpenCV tests)

12. Documentation Plan

Per Module (8 documentation files + 2 config/legal, 40 docs + 10 config/legal = 50 files total)

1. README.md
2. ARCHITECTURE.md (with 5 Mermaid diagrams)
3. API_REFERENCE.md
4. USER_GUIDE.md
5. CONTRIBUTING.md
6. CHANGELOG.md
7. CLAUDE.md / AGENTS.md
8. .env.example
9. VIDEO_COURSE.md (scripts per module)
10. LICENSE

Video Course Scripts (28 total)

- LLMsVerifier Strategy: 5 episodes
- LLMOrchestrator: 5 episodes
- VisionEngine: 5 episodes
- DocProcessor: 5 episodes
- HelixQA Autonomous Session: 8 episodes

Diagrams (25 total, 5 per module)

1. Component diagram
 2. Sequence diagram
 3. Class diagram
 4. State diagram
 5. Flowchart
-

13. Project Management

- GitHub Project with all tasks, descriptions, and status tracking
- GitLab equivalent kept 100% in sync
- Tickets updated as implementation progresses
- New Git submodule repos created via CLI for all new modules